

Week 2 Day 4 — CrewAI Multi-Agent Crew: Roles, Delegation, and Cost Comparison — Write-up

Business task & agent design (Task 1)

Business task: analyze our laptop lineup, benchmark it against a competitor's pricing, and produce a stakeholder-ready recommendation for which laptop to feature this quarter.

Agent	Goal	Backstory	Tools
Product Data Analyst	Extract clear, factual pricing and specification insights from our own laptop lineup.	You are an internal analyst who has studied our product catalog for years. You care about precise numbers and clear price-tier positioning, not marketing language.	get_all_products, get_product_price
Competitive Market Researcher	Benchmark our lineup against the competitor's equivalent products and pricing.	You are a market researcher who tracks competitor pricing closely. You are skeptical of our own marketing claims and focus purely on how we compare to what's actually on the market.	get_competitor_products
Stakeholder Report Writer	Turn the analyst's and researcher's findings into one concise, decision-ready recommendation.	You write for busy stakeholders who want a clear recommendation in under 150 words, not a data dump. You always name a specific product and support the recommendation with the numbers provided.	count_words

Why 3 specialists over 1 generalist: each role has a genuinely different information source (our data vs. the competitor's) and a genuinely different skill (numeric analysis vs. concise business writing) — splitting them keeps each agent's context focused. **Where this ISN'T true:** at this small scale (a handful of products, one short report), a single well-prompted agent could plausibly do all three steps in one pass with less overhead — specialization pays off more as data volume or report complexity grows.

LLM config & tool assignment (Task 2)

LLM config: All agents used gemini/gemini-3.5-flash-lite via CrewAI's native Gemini provider. The Analyst and Researcher ran at temperature=0 (factual, consistent numbers), while the Writer ran at temperature=0.4 (slightly more natural prose). The Hierarchical manager ran at temperature=0. Every agent was capped at max_rpm=4 to stay under the free-tier 5 requests/minute limit.

Tool justification: Analyst → only our internal data tools, deliberately *not* the competitor tool, so it can't blend datasets. Researcher → only the competitor tool, for the same reason in reverse. Writer → only count_words, no data-access tools at all, so it must synthesize from the context=[...] handed to it rather than re-pull raw data.

Format-mismatch fix (Task 3)

The analyst's first expected_output ("A short analysis of our products") produced a loose paragraph of prose, which made it hard for the Writer agent to reliably pull out per-product numbers. Fixed by tightening expected_output to require one bullet per product in a fixed, parseable format:

<name>: \$<price> - <one-line insight>

Confirmed in the run: the Analyst's final answer came back in the required format (e.g. Laptop A: \$650 - Entry-level...), and the Writer correctly reused \$650, \$950, \$1200 and the \$50 / \$50 / \$150 gaps — no numbers dropped or invented. The tightened expected_output fixed the original loose-prose problem.

Sequential vs. Hierarchical (Task 4)

Aspect	Sequential	Hierarchical
Pros	Predictable fixed order; cheapest (no manager overhead); easy to trace	Manager can re-route/re-delegate if a task's output is weak; more robust to unexpected orderings
Cons	No adaptability — a bad analyst output just flows through unchanged	Extra LLM calls for the manager's own planning/review → more tokens, higher latency, less predictable order
When to use	Task order is known/fixed, low budget or latency tolerance, simple pipelines like this one	Task order/assignment may need to adapt at runtime, or a review step before work is 'done' matters

For this specific task (3 clearly-ordered steps, small dataset), sequential is the natural fit — the task order was never ambiguous. In practice, both sequential runs converged on Laptop B, while the hierarchical run recommended Laptop C instead. The manager's extra delegation/review step changed which factors the Writer ultimately weighed, not just the cost.

Actual token usage (Task 5)

Metric	Sequential (run 1)	Hierarchical	Sequential (run 2)
Total tokens	3,126	14,609	8,037
Prompt tokens	2,661	12,770	6,767
Completion tokens	465	1,839	1,270
Successful requests	6	18	14
Final recommendation	Laptop B	Laptop C	Laptop B

The hierarchical run cost roughly 4.7× more tokens (14,609 vs 3,126) and 3× more requests (18 vs 6) than sequential run 1 to reach its answer. The extra cost comes from the Crew Manager's own planning, delegation, and review reasoning layered on top of the same underlying analyst/researcher/writer work.

The runs did NOT all converge on the same recommendation: both sequential runs (run 1 and run 2) recommended Laptop B, while the hierarchical run recommended Laptop C. This is a real difference in output, not just a cost difference — the manager's re-framing of the task (emphasizing Laptop C's price advantage over its closest competitor and higher margin potential) changed which factors the Writer ultimately weighed most heavily.

Note: sequential run 2 unexpectedly ran through the Crew Manager as well — the execution log for this run shows every task handled by the 'Crew Manager' agent via `delegate_work_to_coworker`, the same pattern seen in the hierarchical run, even though `sequential_crew` was explicitly built with `process=Process.sequential`. This appears to be a side effect of reusing the same Agent objects (analyst, researcher, writer) across both `sequential_crew` and `hierarchical_crew` — wiring them up for the hierarchical run appears to leave state behind that persists into later runs. This is why sequential run 2's request count (14) sits between sequential run 1 (6) and hierarchical (18): it paid for manager overhead despite being a 'sequential' run in name. A cleaner design would build separate Agent instances for the hierarchical crew instead of reusing the sequential ones.

Success criteria & evaluation (Task 5)

- **Factual grounding** — does the final recommendation cite a real price figure from each of the analyst's and researcher's outputs, not an invented number?
- **Completeness** — does it name one specific product and address both our internal pricing AND the competitive comparison?
- **Tone/length** — is it a single stakeholder-appropriate paragraph, under 150 words, ending with a clear one-sentence recommendation?

Run	Factual grounding	Completeness	Tone/length
Sequential run 1 (Laptop B, 86 words)	5 — cites \$650, \$950, \$1200, and the \$50 gap vs Rival X2	4 — names one product, but only benchmarks against one competitor (Rival X2), not all three	5 — 86 words, concise, clear recommendation
Hierarchical (Laptop C, 92 words)	5 — cites \$1200, \$1350, \$650, \$700, \$950, \$900	5 — names one product, uses data from both agents	5 — 92 words, concise, clear recommendation
Sequential run 2 (Laptop B, 71 words)	5 — cites \$950, \$900, \$650, \$700, \$1200, \$1350	5 — names one product, uses data from both agents	5 — 71 words, very concise, clear recommendation

Note: all three runs scored cleanly — each cited real prices, named one specific product, and stayed within the 150-word limit (86, 92, and 71 words respectively). The bigger difference was not quality but outcome and cost: the two sequential runs converged on Laptop B in 6 and 14 requests respectively (the second run's higher count is the Crew Manager side-effect noted above), while the hierarchical run used 18 requests and reached a different pick (Laptop C) — showing that the manager's added reasoning step doesn't just cost more, it can change the actual recommendation.

Was the crew worth it? (Task 5)

For this specific task, the 3-agent sequential crew was the sensible choice: it produced a clear, grounded, correctly-formatted recommendation (Laptop B) at roughly $4.7\times$ lower token cost than the hierarchical version (3,126 vs 14,609 tokens; 6 vs 18 requests), and it was consistent across both sequential runs. The hierarchical crew spent far more tokens and reached a different conclusion (Laptop C) — not obviously a better one, since both picks are defensible on the same underlying data. For a task this small (3 products, 2 data sources, one short report), that overhead isn't clearly justified; a single well-prompted agent could plausibly have matched the sequential crew's result with fewer than 6 requests. The 3-agent split becomes clearly worth it once data sources, step count, or report complexity grow beyond what one prompt can hold.